PEMROGRAMAN PERANGKAT BERGERAK

FTEIC – ITS
Teknik Informatika
Surabaya



KULIAH PRAKTIS ANDROID | PERTEMUAN 5

Pemrograman Perangkat Bergerak

Disusun oleh Tim Dosen Pengampu PPB

Departemen Teknik Informatika, FTEIC – ITS

Agenda Kuliah

1. Recap: Aplikasi yang Telah Dibangun pada P1–P4
2. ListView & ArrayAdapter — Menampilkan Data yang Banyak
3. View Recycling — Kenapa ListView Tetap Ringan
4. Custom ArrayAdapter & ViewHolder Pattern untuk Data Kontak
5. Input Dialog — Meminta Data Baru dari Pengguna
6. Mini-Proyek: Aplikasi Buku Kontak Sederhana

Input, Proses, Output — Pertemuan 5

INPUT

- Data kontak dari pengguna (nama, nomor HP) lewat EditText di dalam dialog
- Array/List sementara di memori (ArrayList)
- Data lama tersimpan di SQLite (dari kebiasaan Pertemuan 4)

PROSES

- ArrayAdapter/Custom Adapter mengonversi data menjadi baris ListView
- AlertDialog + LayoutInflater menampilkan formulir input di atas layar
- SQLiteDatabase menyimpan kontak baru & memuat kontak lama saat onCreate

OUTPUT

- Daftar kontak yang bisa di-scroll di layar
- Kontak baru langsung muncul di list setelah dialog ditutup
- Data tetap ada walau aplikasi ditutup dan dibuka kembali

Langkah Ringkas: Alur Kerja Hari Ini

INPUT

- Tampilkan daftar sederhana dengan ListView + ArrayAdapter
- Buat model data Kontak (nama, noHp) dan Custom Adapter untuk itu

PROSES

- Percepat Custom Adapter dengan ViewHolder Pattern
- Buat Input Dialog (AlertDialog + LayoutInflater) untuk menambah kontak

OUTPUT

- Sambungkan tombol Tambah → Dialog → Adapter → tampil di ListView
- Hubungkan semuanya ke SQLite agar data tidak hilang

Analogi Sehari-hari: ListView & Adapter itu seperti Mesin Cetak

Bayangkan proses mencetak 500 undangan dengan nama tamu yang berbeda-beda:

- Tidak perlu menggambar ulang 500 undangan satu per satu dengan tangan.
- Cukup dibuat satu template desain (posisi logo, posisi nama, posisi alamat) — ini seperti item_kontak.xml, layout satu baris list.
- Mesin cetak mengambil template itu lalu "menyetempelnya" berulang kali, setiap kali dengan data nama yang berbeda — ini persis peran Adapter.
- Mesin cetak juga tidak butuh 500 unit mesin fisik berbeda — ia memakai ulang mekanisme cetak yang sama untuk setiap lembar — ini persis View Recycling pada ListView.

Satu template + satu mesin + banyak data = ribuan tampilan serupa tapi berbeda isi, dibuat dengan sangat efisien.

Recap: Aplikasi yang Telah Dibangun pada P1–P4

Selama empat pertemuan terakhir, fondasi Android telah dibangun secara bertahap:

- P1 — Activity & IDE: satu Activity = satu layar; setContentView() menghubungkan Java ke layout XML.
- P2 — Layout & Event: LinearLayout/RelativeLayout, widget dasar, dan menangani onClick pengguna.
- P3 — Intent & Multi-Layar: berpindah antar Activity, mengirim data dengan Intent.putExtra().
- P4 — Penyimpanan Data: SharedPreferences untuk data kecil, SQLite untuk data terstruktur yang persisten.

Pada pertemuan ini, seluruh materi digabungkan menjadi satu aplikasi nyata: Buku Kontak Sederhana — layar dengan daftar dinamis, dialog input, dan penyimpanan permanen.

1. Mengapa Butuh ListView?

Bayangkan aplikasi kontak dengan 200 nama. Menaruh 200 TextView secara manual di XML jelas tidak masuk akal — borosnya memori, dan datanya pun tidak tetap (bisa bertambah kapan saja).

ListView adalah widget yang menampilkan koleksi data secara vertikal dan dapat di-scroll — pola tampilan yang sangat umum di aplikasi mobile (kontak, chat, berita, produk belanja).

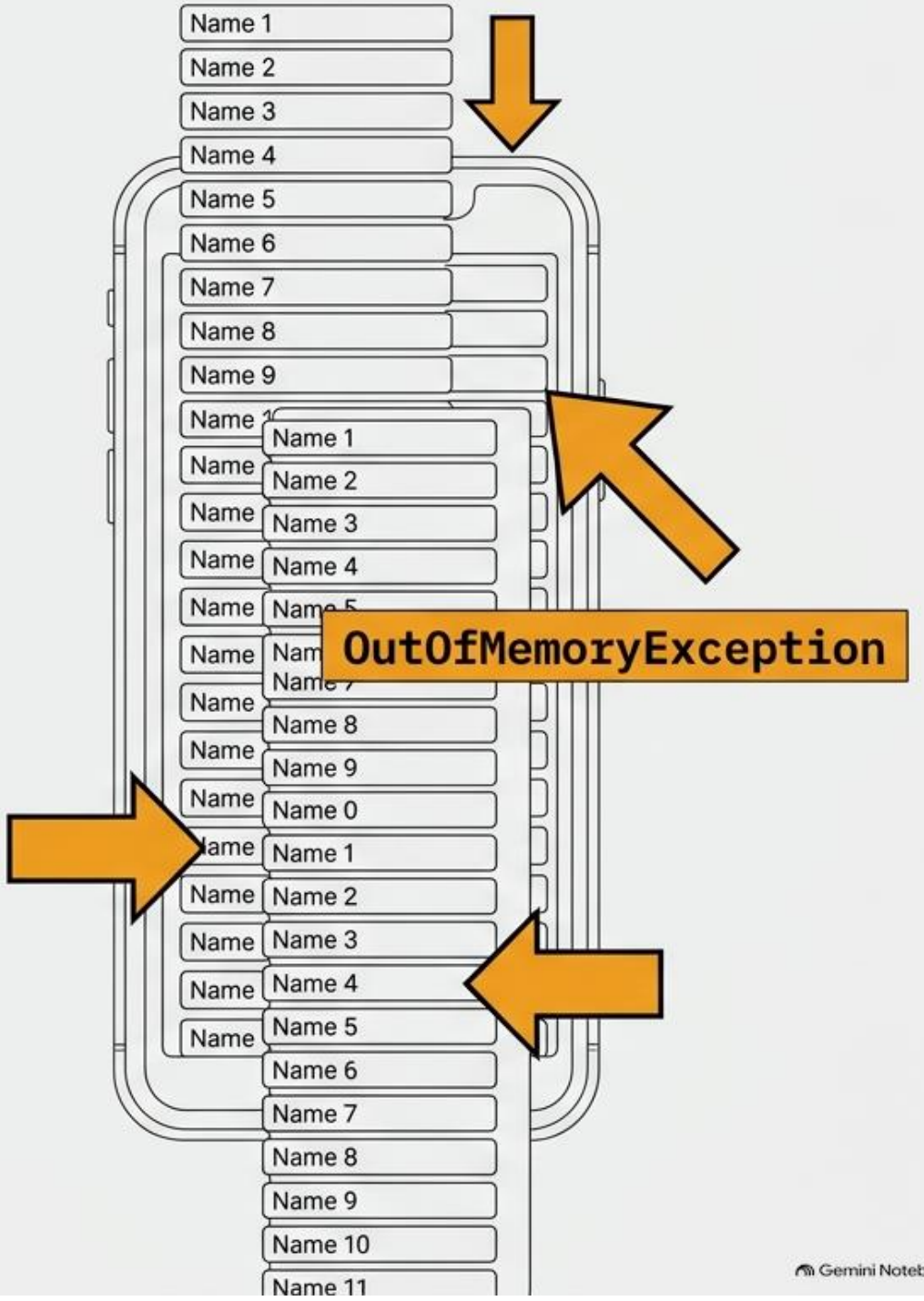
- Pengguna men-scroll daftar, lalu memilih satu item.
- Memilih satu item sering memicu layar detail — di sinilah Intent (Pertemuan 3) biasanya dipakai untuk berpindah ke Activity lain membawa data item terpilih.
- ListView tidak tahu cara mengubah data mentah menjadi tampilan — tugas itu diserahkan ke sebuah objek perantara bernama Adapter.

Keterbatasan Antarmuka Statis

Bayangkan Anda harus merancang aplikasi buku kontak yang memiliki 200 nama di dalamnya. Menaruh 200 elemen XML statis secara manual di dalam struktur file tata letak jelas bukan solusi yang masuk akal maupun berkelanjutan. Pendekatan statis ini sangat **boros** memori karena sistem harus memuat ratusan objek ke dalam RAM **secara bersamaan meskipun sebagian besar tidak terlihat di layar pada satu waktu tertentu**.

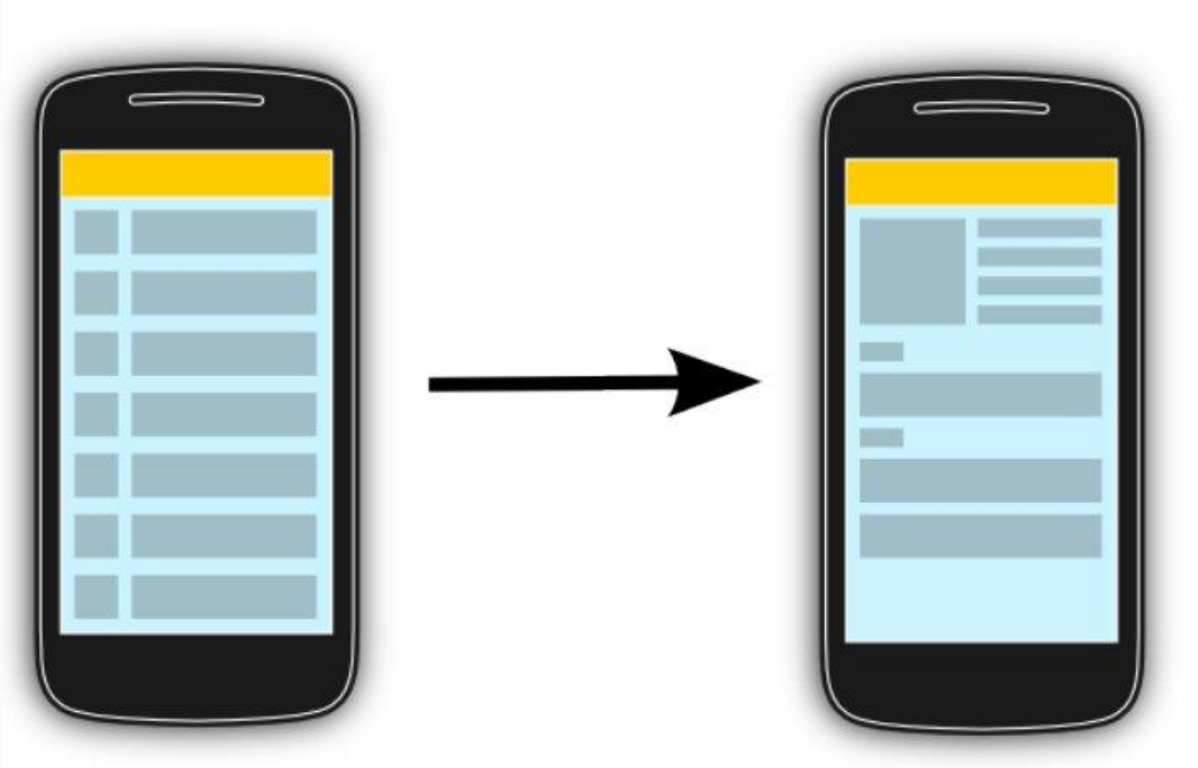
Selain itu, **data pada aplikasi dunia nyata selalu bersifat dinamis**—jumlah baris dapat bertambah atau berkurang kapan saja dari database.

Oleh karena itu, kita membutuhkan sebuah komponen khusus, yaitu *ListView*, yang bertugas mengelola daftar panjang secara cerdas tanpa menyebabkan perangkat kehabisan memori atau mengalami *crash* mendadak.



Gemini Notebook

— Konsep: Dari Daftar ke Detail



Catatan Istilah: Adapter

Sebelum lanjut, satu istilah kunci hari ini:

Adapter: objek perantara yang menjembatani sumber data (mis. ArrayList) dengan tampilan visual (ListView). Ia bertugas mengubah setiap item data menjadi satu baris View di layar.

Tanpa Adapter, ListView tidak tahu apa yang harus ditampilkan — ia hanya kerangka kosong yang menunggu diisi.

Ada beberapa jenis Adapter: ArrayAdapter (dari ArrayList), CursorAdapter (langsung dari hasil query SQLite). Pembahasan pertemuan ini difokuskan pada ArrayAdapter.

Konsep Adapter: Analogi Mesin Cetak

Konsep dasar dari arsitektur daftar di Android tidak menugaskan ListView untuk merender data secara langsung ke layar. ListView tidak tahu cara mengubah data mentah menjadi tampilan visual; tugas krusial ini diserahkan kepada sebuah objek perantara ceral yang krusal ini diserahkan kepada sebuah objek perantara cerdas yang disebut *Adapter*.

Bayangkan proses mencetak ratusan surat undangan: file XML adalah template desain kosong Anda, array data mentah di memori adalah tinta nama-nama tamunya, dan *Adapter* adalah mesin cetak mekanisnya. Mesin cetak ini akan mengambil template kosong, menyetempelnya dengan data dari memori, dan meletakkannya di atas kertas Anda berulang kali. Tanpa adanya jembatan dari Adapter ini, ListView hanyalah sebuah kerangka kosong di layar yang sama sekali tidak memiliki konten fungsional.



Data Mentah
(Tinta)

Adapter
(Mesin Cetak)

Gemini Notebook

2. ArrayAdapter — Jembatan antara Data dan Tampilan

ArrayAdapter adalah jenis Adapter paling sederhana: ia mengubah ArrayList objek menjadi kumpulan View yang dimuat ke dalam ListView.

ArrayAdapter mengatur dua hal:

- Array/List mana yang menjadi sumber data.
- Bagaimana setiap item dalam array itu diubah menjadi satu View (satu baris tampilan).

android.R.layout.simple_list_item_1 adalah layout baris bawaan Android — hanya satu TextView sederhana per baris.

```
ArrayAdapter<String> itemsAdapter =
    new ArrayAdapter<String>(this, android.R.layout.simple_list_ite
      m_1, listNama);
```

Catatan Notasi: Tipe Generic ArrayAdapter

Tanda kurung siku `` yang menempel pada nama kelas adalah generic type di Java — cara memberi tahu kompiler jenis data apa yang akan dikelola kelas tersebut.

- ArrayAdapter = "ArrayAdapter ini khusus mengelola data bertipe String".
- Selanjutnya akan ditulis ArrayAdapter = "ArrayAdapter ini khusus mengelola objek Kontak".
- Manfaatnya: kompiler bisa mendeteksi kesalahan tipe data lebih awal (mis. mencoba memasukkan angka ke adapter yang seharusnya berisi String), sebelum aplikasi dijalankan.

3. Cara Kerja View Recycling

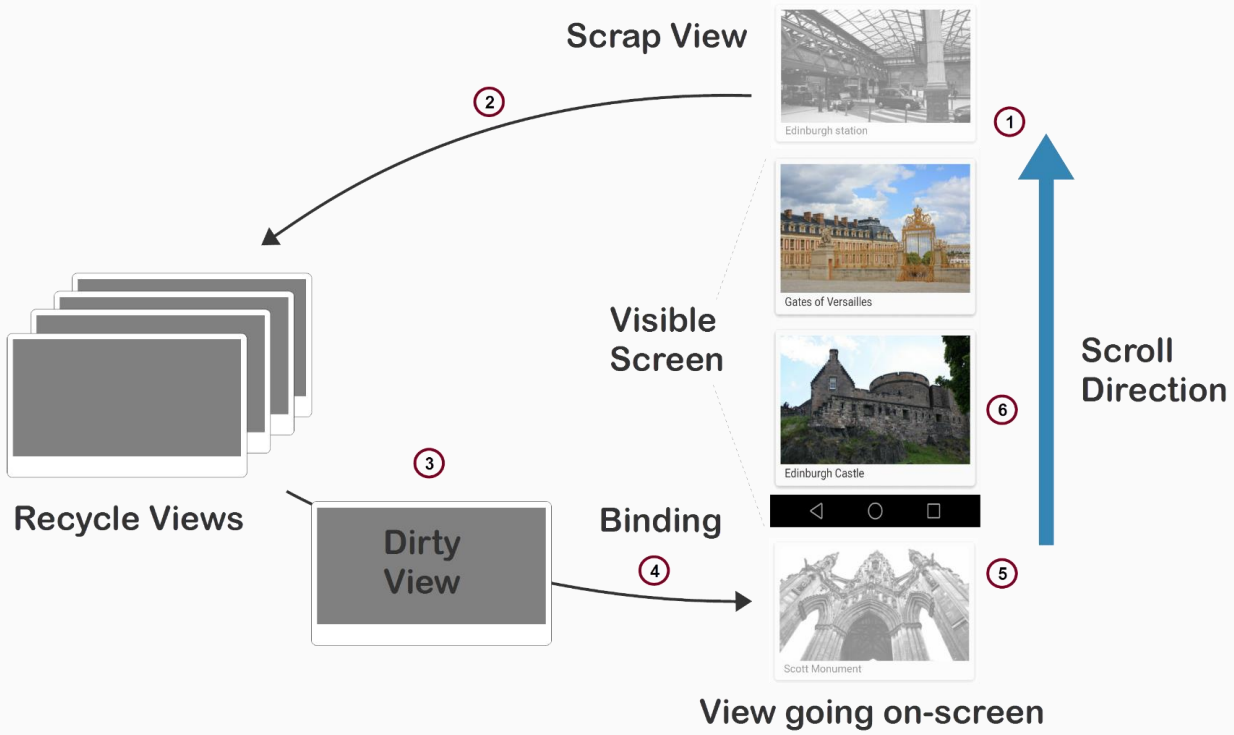
View Recycling adalah alasan utama ListView tetap ringan meski datanya ribuan item.

Saat ListView dipasangi Adapter, ia hanya membuat baris View secukupnya untuk memenuhi tinggi layar (biasanya ~7 baris terlihat) — tidak 1000 baris sekaligus untuk 1000 data.

Saat pengguna men-scroll, baris yang keluar dari layar tidak dibuang, melainkan disimpan di memori dan dipakai ulang (didaur ulang) untuk menampilkan data baris berikutnya yang baru masuk ke layar. Karena itu, walau datanya 1000 item, hanya sekitar 7 objek View saja yang pernah dibuat sepanjang hidup aplikasi.

▶ *Catatan Istilah — View Recycling: teknik memakai ulang objek View yang sudah tidak terlihat di layar untuk menampilkan data lain, alih-alih membuat objek View baru setiap saat. Ini menghemat memori dan mempercepat scrolling.*

— Diagram: Mekanisme View Recycling



Krisis Memori & Konsep View Recycling

Masalah 10.000 Item:

Saat Anda memiliki 10.000 item di dalam database, menginisialisasi 10.000 objek tampilan secara instan akan memicu eksekusi *OutOfMemory* yang mematikan aplikasi seketika.

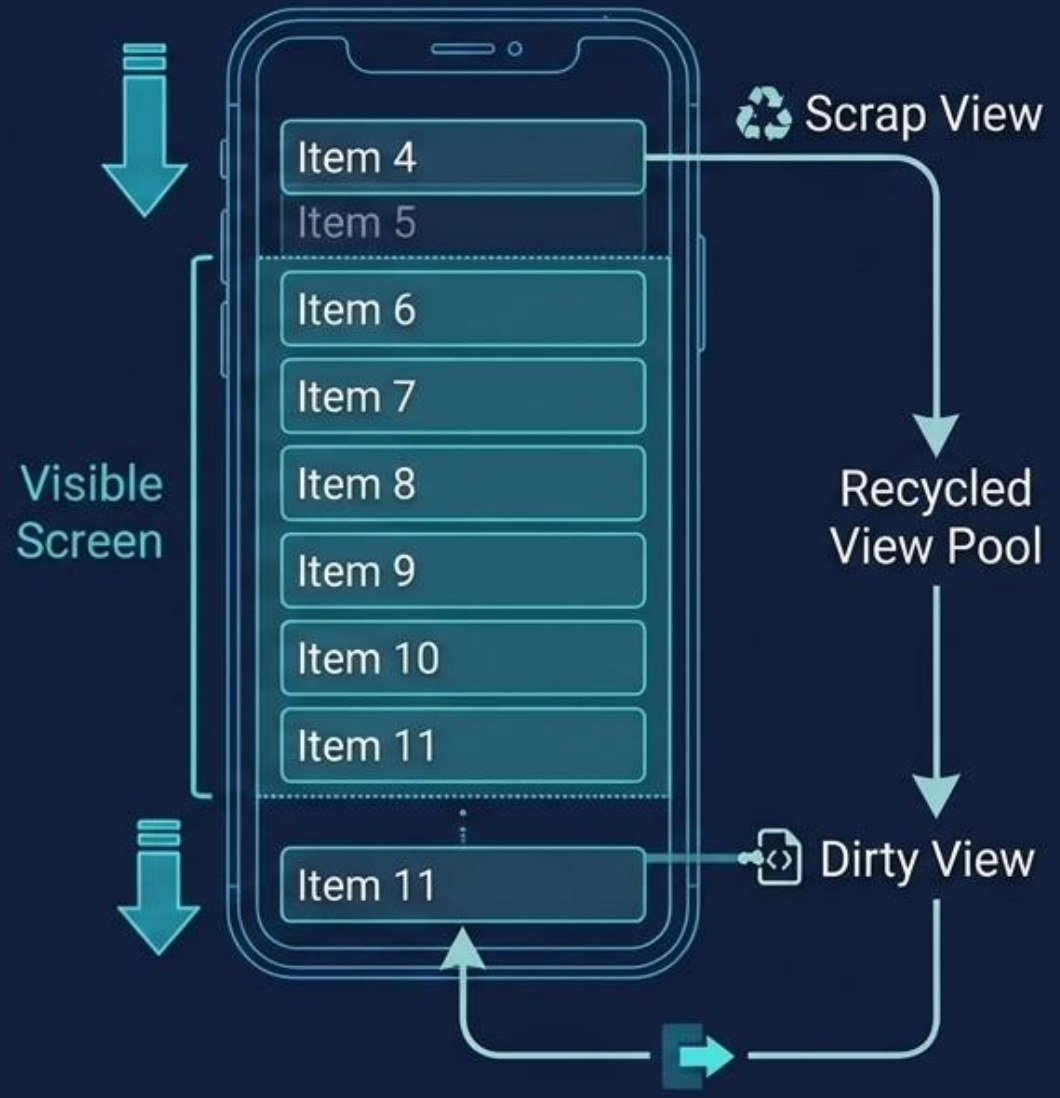
Solusi Daur Ulang:

Android menyelesaikan masalah fundamental ini melalui mekanisme yang disebut *View Recycling* atau daur ulang tampilan. Saat sebuah ListView dipasangi Adapter, ia hanya akan memproduksi objek *View* secukupnya untuk memenuhi tinggi layar (misalnya hanya 7 hingga 10 baris yang tampak saat itu). Ketika pengguna melakukan *scroll* ke bawah, baris teratas yang keluar dari batas layar tidak akan dihancurkan oleh sistem *Garbage Collector*, melainkan disimpan secara cerdas dalam memori sementara dan dibawa ke bagian bawah untuk dipakai ulang. Tampilan usang tersebut kemudian diisi dengan data baru, sehingga menghemat konsumsi memori sistem operasi secara sangat drastis.

Gemini Notebook

Anatomi View Recycling

- **Visible Screen**: Objek View yang saat ini terlihat oleh pengguna (~7 baris).
- **Scrap View**: Objek yang terdorong keluar layar saat scroll.
- **Dirty View**: Objek hasil daur ulang yang diserahkan kembali ke Adapter untuk diisi (di-bind) dengan data item selanjutnya.



Gemini Notebook

4. Praktik: ListView Sederhana — Layout XML

Buat proyek baru dan tambahkan ListView ke activity_main.xml:

Satu ListView saja sudah cukup untuk menampilkan daftar — sisanya diatur lewat kode Java.

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/a
  ndroid"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ListView
        android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

</RelativeLayout>
```

Praktik: ListView Sederhana — MainActivity.java

setAdapter() adalah baris kunci: ia memasangkan Adapter (sumber data + cara menampilkan) ke ListView.

```
public class MainActivity extends AppCompatActivity {
    private ListView lv;
    private ArrayAdapter<String> adapterNama;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        lv = findViewById(R.id.listView);
        String[] nama = {"Agus", "Budi", "Cica", "Dani", "Eka", "Fa
          ni", "Gina"};
        ArrayList<String> listNama = new ArrayList<>(Arrays.asList(
          nama));

        adapterNama = new ArrayAdapter<>(this,
                android.R.layout.simple_list_item_1, listNama);
        lv.setAdapter(adapterNama);
    }
}
```




— Hasil Nyata: ListView Sederhana Berjalan



Gambar 3. Hasil menjalankan kode di atas pada emulator — ListView menampilkan daftar item yang dapat di-scroll, masing-masing baris memakai layout bawaan simple_list_item_1.

Praktikum P05 (Bagian 1)

Tugas Langsung (15 menit):

- Buat proyek baru Empty Views Activity, bahasa Java.
- Tambahkan ListView ke activity_main.xml seperti contoh di atas.
- Isi MainActivity.java dengan array berisi 7 nama teman sekelas.
- Jalankan di emulator, pastikan daftar tampil dan bisa di-scroll.
- Tambahkan 10 nama lagi ke array agar daftar cukup panjang untuk di-scroll.

▶ *Ekspektasi: ListView menampilkan daftar nama, dapat di-scroll naik-turun dengan lancar.*

5. Dari String ke Objek: Model Data Kontak

ArrayAdapter cukup untuk teks polos. Tapi kontak nyata butuh dua informasi: nama dan nomor HP, sehingga diperlukan model data tersendiri.

Kelas Kontak ini murni "kotak penyimpanan data" — tidak tahu apa-apa soal tampilan. Itu tugas Adapter.

```
public class Kontak {
    private String nama;
    private String noHp;

    public Kontak(String nama, String noHp) {
        this.nama = nama;
        this.noHp = noHp;
    }

    public String getNama() { return nama; }
    public void setNama(String nama) { this.nama = nama; }

    public String getNoHp() { return noHp; }
    public void setNoHp(String noHp) { this.noHp = noHp; }
}
```

Membangun Data Model (POJO)

`Kontak.java` — Wadah Penyimpanan Murni

```java
public class Kontak {
    private String nama;
    private String noHp;

    // Konstruktor
    public Kontak(String nama, String noHp) {
        this.nama = nama;
        this.noHp = noHp;
    }

    // Getter dan Setter
    public String getNama() { return nama; }
    public void setNama(String nama) { this.nama = nama; }

    public String getNoHp() { return noHp; }
    public void setNoHp(String noHp) { this.noHp = noHp; }
}
```

Inisialisasi Data

Akses Memori

Gemini Notebook

Layout Baris Kontak: item_kontak.xml

Satu baris ListView kontak butuh dua TextView (nama & nomor HP), sehingga layout simple_list_item_1 bawaan tidak lagi dapat dipakai. Berikut layout kustom res/layout/item_kontak.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/and
  roid"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="12dp">

    <TextView
        android:id="@+id/tvNama"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textStyle="bold"
        android:textSize="16sp" />

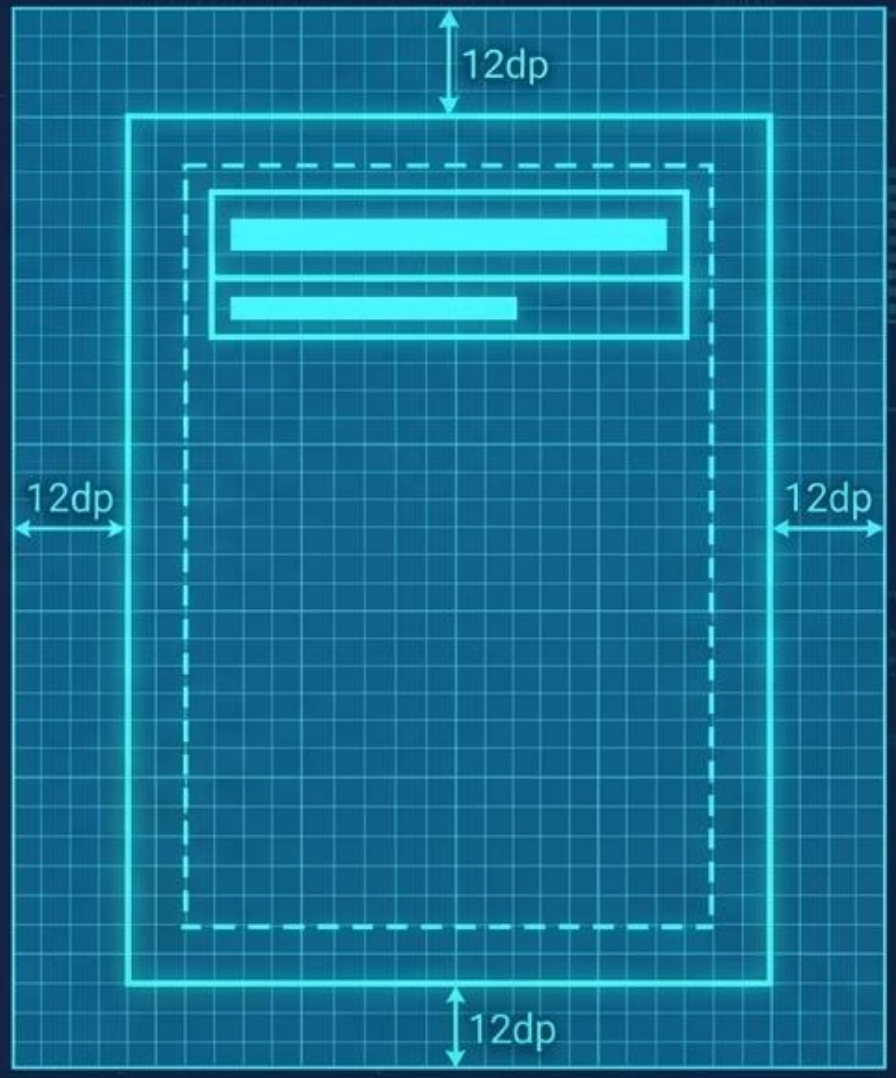
    <TextView
        android:id="@+id/tvNoHp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="14sp" />
</LinearLayout>
```




Solved Case 2: Blueprint Layout Custom XML

`res/layout/item_kontak.xml`

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/an
    android:layout_width="match_parent" android:layout_height="wra
    android:orientation="vertical" android:padding="12dp">
    <TextView android:id="@+id/tvNama"
        android:layout_width="wrap_content" android:layout_height=
        android:textStyle="bold" android:textSize="16sp" />
    <TextView android:id="@+id/tvNoHp"
        android:layout_width="wrap_content" android:layout_height=
        android:textSize="14sp" />
</LinearLayout>
```



Gemini Notebook

6. Custom ArrayAdapter untuk Kontak

Karena layout baris sudah dibuat sendiri, ArrayAdapter bawaan tidak cukup — diperlukan Adapter kustom dengan menurunkan (extends) ArrayAdapter, dan mengisi getView() — metode yang bertugas mengubah satu objek Kontak menjadi satu View baris.

getItem(position) sudah tersedia otomatis dari ArrayAdapter — ia mengambil data pada posisi tertentu dari list yang dikelola.

```
public class KontakAdapter extends ArrayAdapter<Kontak> {

    public KontakAdapter(Context context, int resource, List<Kontak
      > objects) {
        super(context, resource, objects);
    }

    @Override
    public View getView(int position, View convertView, ViewGroup p
      arent) {
        Kontak dataKontak = getItem(position);
        if (convertView == null) {
            convertView = LayoutInflater.from(getContext())
                    .inflate(R.layout.item_kontak, parent, false);
        }
        TextView tvNama = convertView.findViewById(R.id.tvNama);
        TextView tvNoHp = convertView.findViewById(R.id.tvNoHp);
        tvNama.setText(dataKontak.getNama());
        tvNoHp.setText(dataKontak.getNoHp());
        return convertView;
    }
}
```




Custom ArrayAdapter & Inflasi View

```
@Override
public View getView(int position, View convertView, ViewGrou
    // Proses inflasi XML menjadi Java View
    // dan penyuntikan data terjadi di sini
}
```

Karena kita telah merancang susunan *layout* baris sendiri secara spesifik, kelas `ArrayAdapter` bawaan tidak lagi sanggup memetakannya secara otomatis. Kita wajib membuat *Adapter* kustom yang mewarisi (*extends*) `ArrayAdapter` dan menulis ulang metode krusial getView(). Metode ini mewakili siklus hidup pembuatan satu baris tunggal dalam daftar. Di dalam metode ini, kita menggunakan komponen internal LayoutInflater yang bertindak sebagai mesin pembangun; ia membaca teks statis dari berkas XML dan meledakkannya (meng-*inflate*) menjadi objek Java tipe View` yang benar-benar hidup di memori RAM. Objek yang telah hidup inilah yang nantinya akan disuntik dengan teks dari model data POJO dan dikembalikan ke antarmuka utama untuk ditampilkan kepada pengguna akhir.

Gemini Notebook

Melekatkan Adapter Kustom ke ListView

kAdapter.add(...) menambah data dan otomatis menyegarkan tampilan ListView — tidak diperlukan pemanggilan apa pun secara manual untuk me-refresh layar.

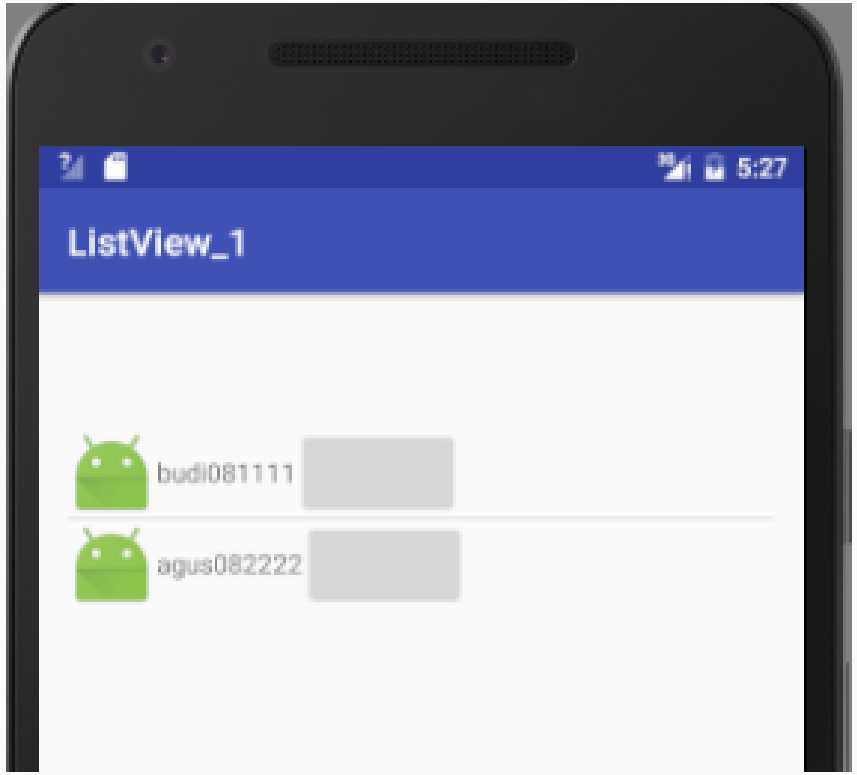
```
public class MainActivity extends AppCompatActivity {
    private ListView lv;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        lv = findViewById(R.id.listView);
        ArrayList<Kontak> listKontak = new ArrayList<>();
        KontakAdapter kAdapter = new KontakAdapter(this, 0, listKon
          tak);
        lv.setAdapter(kAdapter);

        kAdapter.add(new Kontak("Budi", "081111"));
        kAdapter.add(new Kontak("Agus", "082222"));
    }
}
```

— Hasil Nyata: Custom Adapter Menampilkan Objek Kontak



Gambar 4. Hasil KontakAdapter yang baru dibuat — setiap baris kini dirender oleh getView() dari layout item_kontak.xml, bukan lagi simple_list_item_1 bawaan Android.

Catatan Istilah: ViewHolder Pattern

getView() di atas sudah bekerja, tapi ia memanggil findViewById() setiap kali satu baris dibuat/didaur ulang — dan findViewById() tergolong operasi yang relatif lambat karena harus menelusuri struktur View XML.

ViewHolder Pattern: teknik menyimpan hasil findViewById() di dalam sebuah objek kecil (ViewHolder) yang "ditempel" pada View baris menggunakan setTag(), sehingga findViewById() hanya dipanggil sekali per baris yang baru dibuat — bukan setiap kali baris didaur ulang.

Dikombinasikan dengan View Recycling, ini membuat ListView tetap mulus di-scroll walau isinya ribuan data.

7. Meningkatkan Performa: ViewHolder Pattern

```
public class KontakAdapter extends ArrayAdapter<Kontak> {

    private static class ViewHolder {
        TextView tvNama;
        TextView tvNoHp;
    }

    public KontakAdapter(Context context, int resource, List<Kontak> objects) {
        super(context, resource, objects);
    }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        Kontak dataKontak = getItem(position);
        ViewHolder holder;

  ...
```




Optimasi Mutlak: ViewHolder Pattern

`findViewById()`



Memicu lag karena harus menelusuri hierarki struktur XML berulang kali.

Meskipun *View Recycling* berhasil memangkas pembuatan objek baru secara masif, proses ini masih menyimpan satu kelemahan kinerja yang fatal: fungsi `findViewById()` tetap dipanggil setiap kali sebuah baris didaur ulang. Fungsi ini sangat berat karena memaksa CPU perangkat menelusuri hierarki struktur XML secara terus-menerus selama pengguna melakukan *scroll* secara cepat.

Untuk mengatasi *lag* visual ini, industri menetapkan standar penggunaan *ViewHolder Pattern*. Pola arsitektur ini memanfaatkan kelas statis internal untuk menyimpan (melakukan *cache*) referensi komponen visual secara permanen di dalam memori. Referensi yang disimpan ini kemudian ditempelkan secara langsung pada objek baris menggunakan fungsi setTag(), sehingga penelusuran XML yang memakan kinerja CPU hanya terjadi tepat satu kali untuk setiap objek fisik yang diproduksi.

Gemini Notebook

Matriks Komparasi Performa Render

Pendekatan	Konsumsi RAM	Beban CPU (findViewById)	Kualitas Scroll
Naive (Tanpa Daur Ulang)	Boros (1000+ Objek)	Sangat Tinggi (1000+ kali)	Macet/Crash (OutOfMemory)
View Recycling Dasar	Hemat (~7 Objek)	Sedang (Saat daur ulang)	Patah-patah saat scroll cepat
Recycling + ViewHolder	Sangat Hemat (~7 Objek)	Sangat Rendah (7 kali di awal)	Sangat Mulus (Industry Standard)

Gemini Notebook

Catatan Notasi: static class & setTag()/getTag()

Dua hal baru pada kode sebelumnya:

- private static class ViewHolder { ... } — ini adalah nested class (kelas di dalam kelas), ditandai static karena ia tidak butuh akses ke data instance KontakAdapter luar — ia murni kotak penyimpanan referensi TextView.

- convertView.setTag(holder) — menempelkan objek Java apa pun (di sini ViewHolder) ke sebuah View, seolah memberi "label tersembunyi". getTag() mengambilnya kembali. Ini trik standar Android untuk "mengingat" data terkait sebuah View tanpa mencari ulang dari awal.

- Alur cek if (convertView == null): null berarti baris baru (belum pernah dibuat) → cari & simpan referensi. Sebaliknya berarti baris daur ulang → langsung pakai referensi lama dari getTag().



Solved Case 3: Implementasi Penuh ViewHolder

```java
public class KontakAdapter extends ArrayAdapter<Kontak> {
    private static class ViewHolder { TextView tvNama; TextView tvNoHp; }

    public KontakAdapter(Context context, int resource, List<Kontak> objects) {
        super(context, resource, objects);
    }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        Kontak data = getItem(position);
        ViewHolder holder;
        if (convertView == null) {
            convertView = LayoutInflater.from(getContext()).inflate(R.layout.item_kontak, parent, false);
            holder = new ViewHolder();
            holder.tvNama = convertView.findViewById(R.id.tvNama);
            holder.tvNoHp = convertView.findViewById(R.id.tvNoHp);
            convertView.setTag(holder); // Cache objek ViewHolder
        } else {
            holder = (ViewHolder) convertView.getTag(); // Ambil dari cache
        }
        holder.tvNama.setText(data.getNama());
        holder.tvNoHp.setText(data.getNoHp());
        return convertView;
    }
```

Gemini Notebook

8. Input Dialog — Meminta Data dari Pengguna

Sejauh ini, ListView sudah dapat menampilkan kontak. Selanjutnya: bagaimana pengguna menambah kontak baru tanpa berpindah ke layar lain?

Dialog adalah jendela kecil yang muncul di atas layar, meminta pengguna membuat keputusan atau mengisi informasi tambahan sebelum melanjutkan — tidak memenuhi seluruh layar seperti Activity baru.

- AlertDialog: jenis dialog paling umum di Android, mendukung judul, isi (bisa layout kustom), dan tombol aksi (mis. OK/Batal).
- LayoutInflater: kelas yang "membangun" objek View hidup dari berkas layout .xml — dipakai di sini untuk memasukkan layout formulir kustom ke dalam AlertDialog.

Membuat Input Dialog melibatkan dua langkah: (1) siapkan layout XML formulirnya, (2) inflate layout itu lalu pasang ke AlertDialog.Builder.

Interupsi Dialog & Builder Pattern

Saat pengguna ingin menambahkan kontak baru ke dalam daftar, kita tidak selalu perlu memindahkan mereka secara paksa ke *Activity* atau layar yang sama sekali baru. Kita dapat menggunakan fitur *Dialog*, yaitu sebuah jendela kecil *modal* yang muncul melayang di atas layar saat ini untuk meminta input data kritis tanpa merusak konteks alur kerja pengguna. Di sistem operasi Android, kelas **`AlertDialog`** adalah standar industri untuk kebutuhan interupsi ini.

Karena konstruksi sebuah jendela dialog membutuhkan pengaturan banyak sekali parameter independen—mulai dari teks judul, ikon grafis, susunan tombol utama, hingga tata letak antarmulkuka khusus—Android memanfaatkan arsitektur *Builder Pattern*. Pola desain ini memungkinkan kita merakit parameter elemen grafis secara bertahap (langkah demi langkah) sebelum akhirnya memanggil fungsi penutup untuk menampilkan jendela tersebut ke permukaan layar.



Gemini Notebook

Layout Dialog Tambah Kontak: add_kontak.xml

Layout ini bukan layout Activity — ia tidak pernah dipasang lewat setContentView(), melainkan di-inflate langsung ke dalam dialog.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/and
  roid"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">

    <EditText
        android:id="@+id/etNama"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Nama" />

    <EditText
        android:id="@+id/etNoHp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Nomor HP"
        android:inputType="phone" />

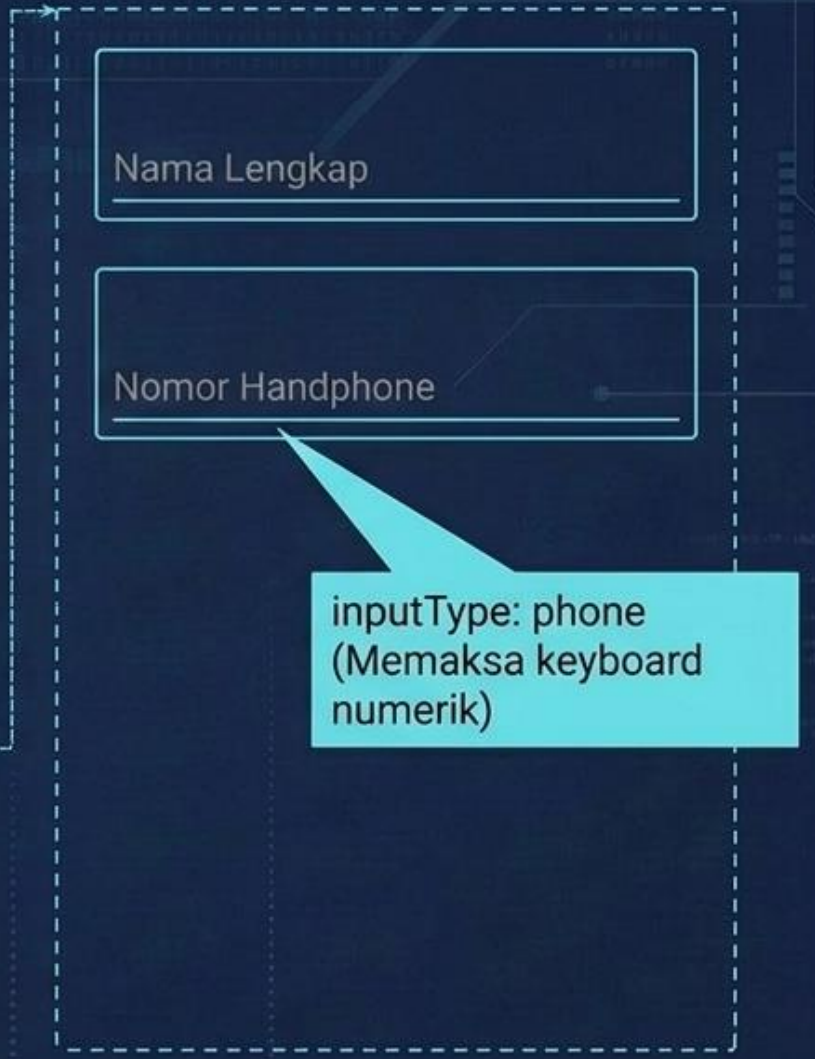
</LinearLayout>
```




Layout Kustom untuk Input Form

`res/layout/add_kontak.xml`

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="wrap_conten
    android:orientation="vertical" android:padding="16dp">
    <EditText android:id="@+id/etNama"
        android:layout_width="match_parent" android:layout_height="wrap_co
        android:hint="Nama Lengkap" />
    <EditText android:id="@+id/etNoHp"
        android:layout_width="match_parent" android:layout_height="wrap_co
        android:hint="Nomor Handphone" android:inputType="phone" />
</LinearLayout>
```



Gemini Notebook

9. Membangun Dialog: LayoutInflater + AlertDialog.Builder

AlertDialog.Builder memakai method chaining: setiap .setXxx(...) mengembalikan Builder itu sendiri, sehingga pemanggilannya bisa disambung .setTitle(...).setView(...).setCancelable(...) berurutan tanpa mengulang nama variabel.

```
private void tampilkanDialogTambah() {
    LayoutInflater li = LayoutInflater.from(this);
    View viewInput = li.inflate(R.layout.add_kontak, null);

    final EditText etNama = viewInput.findViewById(R.id.etNama);
    final EditText etNoHp = viewInput.findViewById(R.id.etNoHp);

    AlertDialog.Builder builder = new AlertDialog.Builder(this);
    builder.setTitle("Tambah Kontak");
    builder.setView(viewInput);
    builder.setCancelable(false);
    // tombol OK/Batal akan ditambahkan berikutnya
    builder.show();
}
```

Solved Case 5 (Bagian A): Merakit Dialog Input

```
private void tampilkanDialogTambah() {
    View viewInput = LayoutInfater.from(this).inflate(R.layout.add_kontak, null);
    final EditText etNama = viewInput.findViewById(R.id.etNama);
    final EditText etNoHp = viewInput.findViewById(R.id.etNoHp);

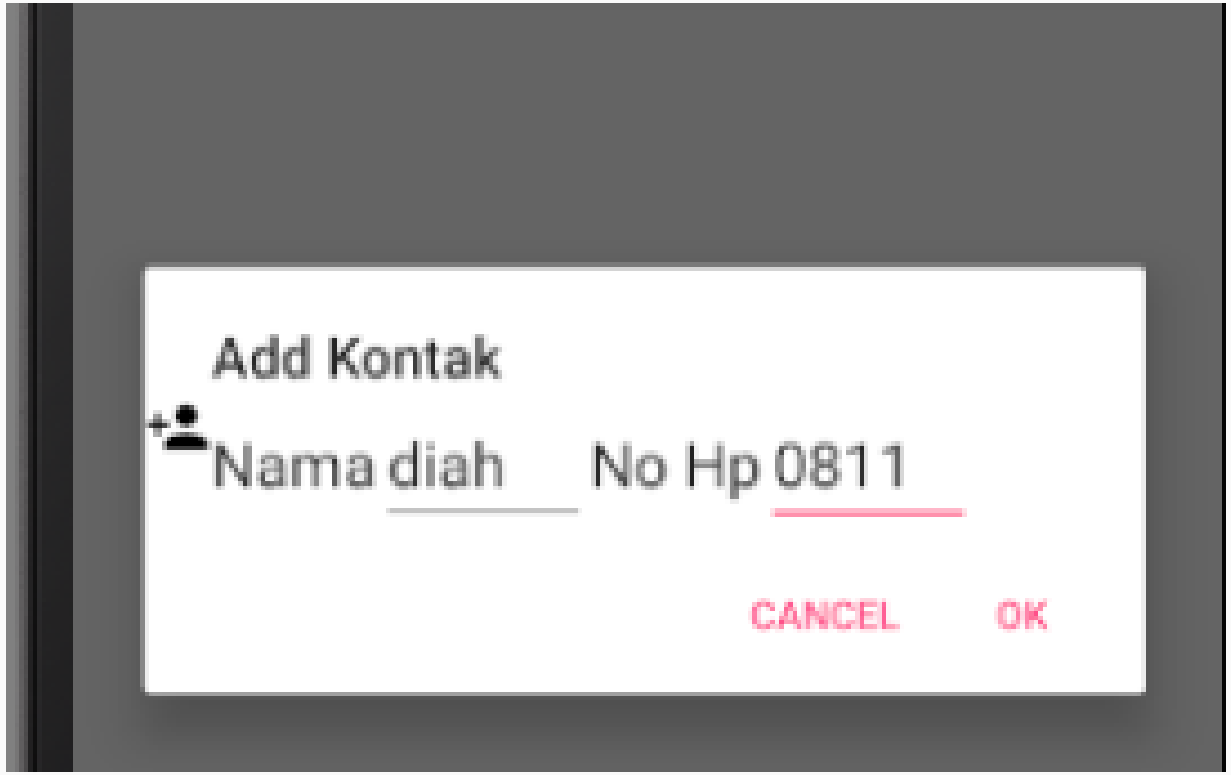
    AlertDialog.Builder builder = new AlertDialog.Builder(this);
    builder.setTitle("Tambah Kontak");
    builder.setView(viewInput);
    builder.setCancelable(false);
    // Tombol Aksi akan ditambahkan di sini
    builder.show();
}
```

Inflasi XML form khusus menjadi View.

Menyuntikkan View kustom ke dalam kerangka Builder Dialog.

Gemini Notebook

— Hasil Nyata: Dialog "Tambah Kontak"



Gambar 5. Tampilan AlertDialog hasil inflate dari add_kontak.xml — dua EditText (Nama, No Hp) mengambang di atas layar utama, lengkap dengan tombol CANCEL/OK.

10. Menangani Tombol OK/Batal

Tombol dialog ditangani lewat DialogInterface.OnClickListener — mirip View.OnClickListener, tapi khusus untuk elemen di dalam dialog.

setPositiveButton = tombol aksi utama (mis. "Simpan"/"OK"), setNegativeButton = tombol pembatalan. dialog.dismiss() menutup dialog setelah aksi selesai; dialog.cancel() menutupnya tanpa aksi.

```
builder.setPositiveButton("Simpan", new DialogInterface.OnClickList
  ener() {
    @Override
    public void onClick(DialogInterface dialog, int which) {
        String nama = etNama.getText().toString();
        String noHp = etNoHp.getText().toString();
        // TODO: simpan ke adapter
        dialog.dismiss();
    }
});

builder.setNegativeButton("Batal", new DialogInterface.OnClickListe
  ner() {
    @Override
    public void onClick(DialogInterface dialog, int which) {
        dialog.cancel();
    }
});
```

Solved Case 4: Aksi Dialog & Klik ListView

1. Aksi Tombol Dialog (Simpan)

```
builder.setPositiveButton("Simpan", new DialogInterface.OnClickListener() {
    @Override
    public void onClick(DialogInterface dialog, int which) {
        simpanKontak(etNama.getText().toString(), etNoHp.getText().toString());
        dialog.dismiss();
    }
});
```

2. Event Listener ListView (Klik Baris)

```
lvKontak.setOnItemClickListener(new AdapterView.OnItemClickListener() {
    @Override
    public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
        Kontak dipilih = kAdapter.getItem(position);
        Toast.makeText(MainActivity.this, dipilih.getNama(), Toast.LENGTH_SHORT).show();
    }
});
```

Gemini Notebook

11. Variabel Lokal vs Variabel Global dalam Dialog

Perhatikan etNama dan etNoHp pada slide sebelumnya dideklarasikan final di dalam method tampilkanDialogTambah(), bukan sebagai field kelas (variabel global).

Ini sengaja: OnClickListener di dalam builder.setPositiveButton(...) adalah kelas anonim yang butuh mengakses etNama/etNoHp — di Java, kelas anonim hanya bisa mengakses variabel lokal yang bertanda final (nilainya tidak berubah setelah diisi). Dengan pola ini, EditText tidak perlu dibuat sebagai variabel global milik MainActivity hanya demi satu dialog — kode jadi lebih rapi dan tidak "mengotori" kelas Activity dengan variabel yang cuma dipakai sesaat.

Tabel: Variabel Global vs Variabel Lokal final

Aspek	Variabel Global (field kelas)	Variabel Lokal final
Tempat deklarasi	Di luar method, level kelas	Di dalam method yang memakainya
Umur hidup	Selama Activity ada	Hanya selama method/dialog berjalan
Risiko	Menumpuk jika banyak dialog dibuat	Otomatis "dibuang" setelah tidak dipakai
Cocok untuk	Data yang dipakai di banyak method	Data yang hanya relevan untuk satu aksi

Rekomendasi: gunakan variabel lokal final untuk kebutuhan sesaat seperti isi dialog — hindari variabel global kecuali benar-benar dibutuhkan di banyak tempat.

Praktikum P05 (Bagian 2)

Tugas Langsung (15 menit):

- Tambahkan tombol "Tampilkan Dialog" ke activity_main.xml proyek Bagian 1.
- Buat add_kontak.xml seperti contoh di atas.
- Tulis tampilkanDialogTambah() lengkap dengan tombol Simpan & Batal.
- Saat tombol Simpan ditekan, tampilkan Toast berisi nama yang diketik.
- Jalankan dan pastikan dialog muncul, bisa diisi, dan bisa dibatalkan.

▶ Ekspektasi: Menekan tombol membuka dialog dengan dua kolom isian; menekan Simpan menampilkan Toast berisi nama yang diketik.



12. Menggabungkan Semuanya: Tombol Tambah → Dialog → ListView

Pada bagian ini, seluruh materi pertemuan disatukan: ListView + Custom Adapter (bagian 4–7) disambungkan ke Input Dialog (bagian 8–11) lewat satu tombol Tambah.

Alih-alih menampilkan Toast, tombol Simpan pada dialog akan memanggil kAdapter.add(...) — dan karena ArrayAdapter otomatis me-refresh tampilan, kontak baru langsung muncul di ListView tanpa kode tambahan apa pun.

Pola arsitekturnya:

- ImageButton di activity_main.xml → memicu tampilkanDialogTambah().
- Dialog menangkap input nama & noHp.
- Tombol Simpan di dialog memanggil simpanKontak(nama, noHp).
- simpanKontak() membungkus data jadi objek Kontak lalu memanggil kAdapter.add(...).

Layout Tombol Tambah: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ImageButton
        android:id="@+id/btnTambah"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentTop="true"
        android:layout_alignParentEnd="true"
        android:layout_margin="12dp"
        android:contentDescription="Tambah Kontak"
        android:src="@android:drawable/ic_input_add" />

    <ListView
  ...
```




Kode: simpanKontak() Menghubungkan Dialog ke Adapter

Ini adalah baris penghubung terpenting pada pertemuan ini: satu pemanggilan kAdapter.add(...) membuat objek Kontak baru langsung tampil sebagai baris baru di ListView — berkat View Recycling + ArrayAdapter yang telah dipelajari, tampilan menyegarkan dirinya sendiri tanpa perlu notifyDataSetChanged() dipanggil manual (karena ArrayAdapter.add() sudah melakukannya secara otomatis).

```
private KontakAdapter kAdapter;

private void simpanKontak(String nama, String noHp) {
    kAdapter.add(new Kontak(nama, noHp));
}
```




Menjembatani Data Dinamis ke UI

Pemanggilan fungsi `add()` pada objek `ArrayAdapter` tidak hanya sekadar memasukkan data ke dalam struktur `ArrayList` di memori, tetapi juga secara otomatis memicu sinkronisasi latar belakang untuk menyegarkan tampilan `ListView` secara instan tanpa perlu intervensi tambahan.

```
private KontakAdapter kAdapter;

private void simpanKontak(String nama, String noHp) {
    // Objek baru masuk ke adapter, layar langsung ter-update
    kAdapter.add(new Kontak(nama, noHp));
}
```

Gemini Notebook

Langkah: Alur Lengkap "Tambah Satu Kontak Baru"

1

Pengguna menekan ImageButton "Tambah" di pojok layar

2

tampilkanDialogTambah() dijalankan → AlertDialog muncul di atas layar

3

Pengguna mengisi etNama dan etNoHp, menekan tombol Simpan

4

OnClickListener tombol Simpan membaca isi kedua EditText

5

simpanKontak(nama, noHp) membungkus data menjadi objek Kontak

6

kAdapter.add(kontak) menambah data — ListView otomatis menampilkan baris baru

13. Dari Sementara ke Permanen: Menghubungkan ke SQLite

Aplikasi Buku Kontak yang dibangun sudah lengkap secara fungsi — tapi datanya masih hilang setiap kali aplikasi ditutup, karena hanya tersimpan di ArrayList di memori.

Ingat kembali Pertemuan 4: SQLiteOpenHelper + SQLiteDatabase untuk penyimpanan data terstruktur yang persisten. Teknik yang sama persis akan dipakai, hanya kali ini datanya langsung ditampilkan lewat ListView.

Ada dua titik sambung yang perlu ditambahkan:

- Saat kontak baru disimpan → insert juga ke tabel kontak di SQLite.
- Saat aplikasi pertama kali dibuka (onCreate) → muat semua data lama dari SQLite ke ListView.

Menyiapkan SQLiteOpenHelper: DbHelper.java

Pola ini sama persis dengan yang dipelajari di Pertemuan 4 — hanya nama tabel & kolomnya disesuaikan untuk data kontak.

```
public class DbHelper extends SQLiteOpenHelper {

    public DbHelper(Context context) {
        super(context, "kontak.db", null, 1);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE IF NOT EXISTS kontak (" +
                "_id INTEGER PRIMARY KEY AUTOINCREMENT, " +
                "nama TEXT, nohp TEXT);");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int ne
      wVersion) {
        // Skema masih versi 1 — belum ada migrasi yang perlu ditan
          gani
    }
}
```




Modifikasi simpanKontak(): Insert ke Database

Perhatikan urutannya: insert ke database dulu, baru tambahkan ke Adapter. Kalau urutan dibalik dan insert gagal, tampilan sudah terlanjur menunjukkan data yang sebenarnya belum tersimpan.

```
private SQLiteDatabase dbKu;

private void simpanKontak(String nama, String noHp) {
    ContentValues nilai = new ContentValues();
    nilai.put("nama", nama);
    nilai.put("nohp", noHp);
    dbKu.insert("kontak", null, nilai);

    kAdapter.add(new Kontak(nama, noHp));
}
```




Memuat Data Saat Aplikasi Dibuka: muatDataDariDatabase()

Dipanggil sekali di akhir onCreate(), setelah kAdapter dipasang ke ListView — sehingga begitu aplikasi dibuka, seluruh kontak yang pernah disimpan langsung tampil kembali.

```
private void muatDataDariDatabase() {
    Cursor cursor = dbKu.rawQuery("SELECT * FROM kontak", null);
    if (cursor.moveToFirst()) {
        do {
            String nama = cursor.getString(cursor.getColumnIndexOrT
              hrow("nama"));
            String noHp = cursor.getString(cursor.getColumnIndexOrT
              hrow("nohp"));
            kAdapter.add(new Kontak(nama, noHp));
        } while (cursor.moveToNext());
    }
    cursor.close();
}
```




14. Kode Lengkap: Aplikasi Buku Kontak (Capstone)

Berikut kode lengkap, bersih, dan siap-dijalankan yang menggabungkan seluruh materi hari ini menjadi satu aplikasi: Buku Kontak Sederhana.

File yang terlibat:

- Kontak.java — model data
- item_kontak.xml — layout satu baris ListView
- KontakAdapter.java — Custom Adapter + ViewHolder Pattern
- DbHelper.java — SQLiteOpenHelper
- activity_main.xml, add_kontak.xml — layout Activity & dialog
- MainActivity.java — menyatukan semuanya

Salin berurutan ke proyek Android Studio (Empty Views Activity, bahasa Java) untuk mencobanya sendiri.

Kode Lengkap (1/5): Kontak.java + item_kontak.xml

Sebelum

```
public class Kontak {
    private String nama;
    private String noHp;

    public Kontak(String nama, String noHp) {
        this.nama = nama;
        this.noHp = noHp;
    }

    public String getNama() { return nama; }
    public String getNoHp() { return noHp; }
}
```

Sesudah

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="wrap_content"
    android:orientation="vertical" android:padding="12dp">
    <TextView android:id="@+id/tvNama" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:textStyle="bold" />
    <TextView android:id="@+id/tvNoHp" android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</LinearLayout>
```

Kode Lengkap (2/5): KontakAdapter.java

```
public class KontakAdapter extends ArrayAdapter<Kontak> {

    private static class ViewHolder {
        TextView tvNama;
        TextView tvNoHp;
    }

    public KontakAdapter(Context context, int resource, List<Kontak> objects) {
        super(context, resource, objects);
    }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        Kontak dataKontak = getItem(position);
        ViewHolder holder;
        if (convertView == null) {
  ...
```




Kode Lengkap (3/5): DbHelper.java + activity_main.xml

Sebelum

```
public class DbHelper extends SQLiteOpenHelper {
    public DbHelper(Context context) { super(context, "kontak.db", null, 1);
      }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE IF NOT EXISTS kontak (" +
                "_id INTEGER PRIMARY KEY AUTOINCREMENT, nama TEXT, nohp TEXT)
                  ;");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldV, int newV) { }
}
```

Sesudah

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent">
    <ImageButton android:id="@+id/btnTambah" android:layout_width="wrap_conte
      nt"
        android:layout_height="wrap_content" android:layout_alignParentEnd="t
          rue"
        android:src="@android:drawable/ic_input_add" />
    <ListView android:id="@+id/lvKontak" android:layout_width="match_parent"
        android:layout_height="match_parent" android:layout_below="@id/btnTam
          bah" />
</RelativeLayout>
```

Kode Lengkap (4/5): add_kontak.xml + MainActivity (onCreate)

Sebelum

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="wrap_content"
    android:orientation="vertical" android:padding="16dp">
    <EditText android:id="@+id/etNama" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:hint="Nama" />
    <EditText android:id="@+id/etNoHp" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:hint="Nomor HP" />
</LinearLayout>
```

Sesudah

```
public class MainActivity extends AppCompatActivity {
    private ListView lvKontak;
    private KontakAdapter kAdapter;
    private SQLiteDatabase dbKu;
    private DbHelper dbHelper;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        lvKontak = findViewById(R.id.lvKontak);
        ImageButton btnTambah = findViewById(R.id.btnTambah);

        ArrayList<Kontak> listKontak = new ArrayList<>();
        kAdapter = new KontakAdapter(this, 0, listKontak);
        lvKontak.setAdapter(kAdapter);

        dbHelper = new DbHelper(this);
        dbKu = dbHelper.getWritableDatabase();

        btnTambah.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) { tampilkanDialogTambah(); }
  ...
```

Kode Lengkap (5/5): MainActivity (dialog, simpan, muat data)

```
    private void tampilkanDialogTambah() {
        View viewInput = LayoutInflater.from(this).inflate(R.layout.add_kontak, null);
        final EditText etNama = viewInput.findViewById(R.id.etNama);
        final EditText etNoHp = viewInput.findViewById(R.id.etNoHp);

        AlertDialog.Builder builder = new AlertDialog.Builder(this);
        builder.setTitle("Tambah Kontak");
        builder.setView(viewInput);
        builder.setPositiveButton("Simpan", new DialogInterface.OnClickListener() {
            @Override
            public void onClick(DialogInterface dialog, int which) {
                simpanKontak(etNama.getText().toString(), etNoHp.getText().toString());
                dialog.dismiss();
            }
        });
        builder.setNegativeButton("Batal", new DialogInterface.OnClickListener() {
...
```




— Hasil Akhir: Aplikasi Buku Kontak Lengkap

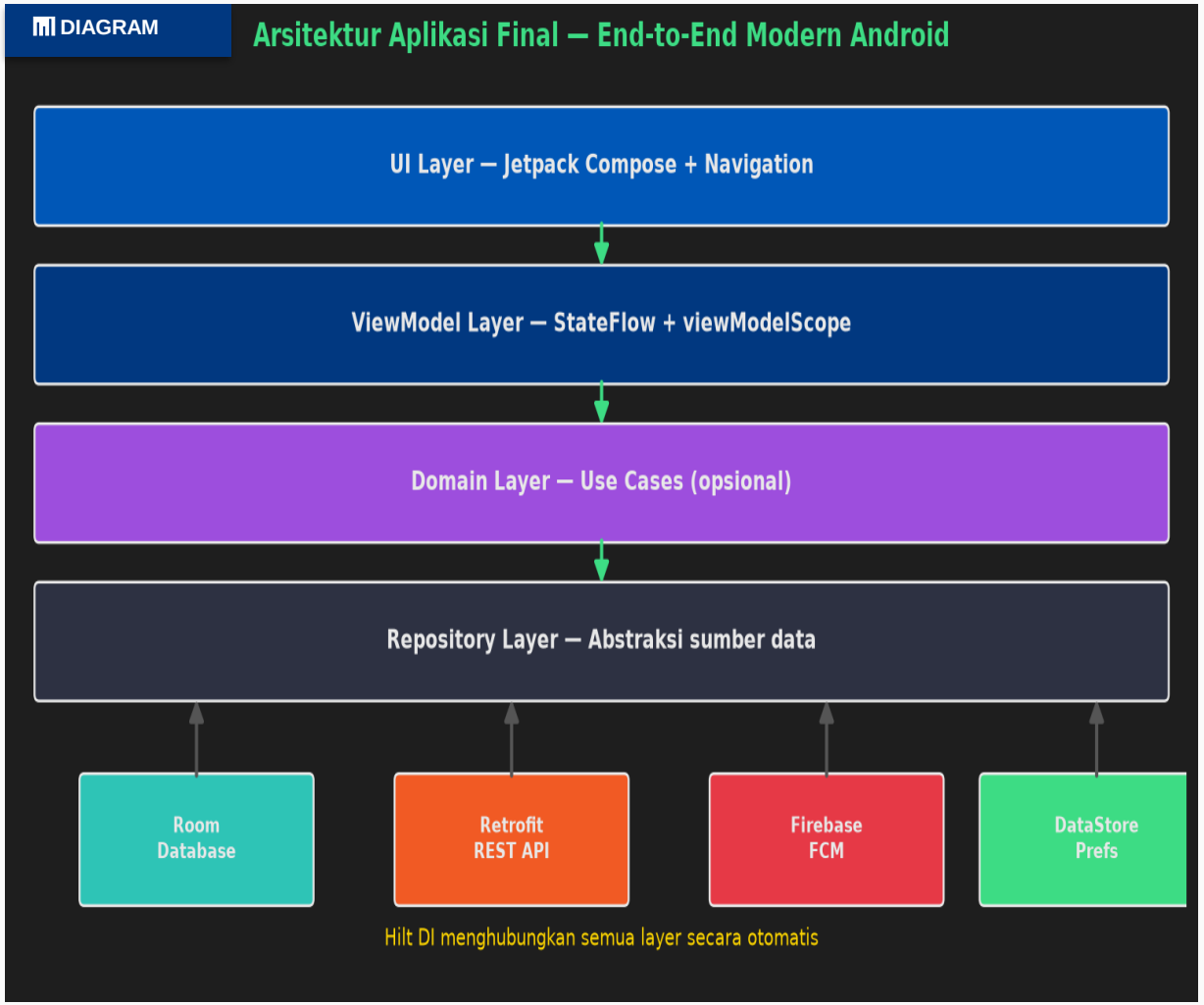


Praktikum P05 (Final)

Tugas Mini-Proyek (30 menit):

- Buat proyek baru, salin kelima berkas kode lengkap di atas apa adanya.
- Sesuaikan nama package pada setiap berkas .java (baris package ...;).
- Jalankan di emulator — tambahkan minimal 3 kontak lewat dialog.
- Tutup total aplikasi (bukan hanya minimize), lalu buka kembali.
- Pastikan ketiga kontak yang ditambahkan masih tampil di ListView.

► Ekspektasi: Kontak yang ditambahkan tetap ada setelah aplikasi ditutup dan dibuka ulang — bukti data sudah tersimpan permanen di SQLite.



Troubleshooting: Masalah & Solusi Umum

Salah satu kesalahan fatal paling umum saat membangun arsitektur dialog kustom adalah munculnya pesan *NullPointerException* ketika memanggil `findViewById`. Hal ini terjadi jika Anda lupa mencari dari objek *View* yang baru saja di-*inflate* (yakni memanggil `viewInput.findViewById()`) dan justru memanggil fungsi milik *Activity* utama, sehingga sistem operasi tidak dapat menemukan `EditText` yang bersembunyi di dalam *layout* dialog.

Kasus kritis lainnya adalah komponen `ListView` yang menolak memperbarui tampilannya setelah isi database berubah. Jika Anda memodifikasi *ArrayList* murni tanpa melewati perantara fungsi milik *Adapter* (seperti `add()` atau `remove()`), sistem perantara tidak akan menyadari adanya mutasi data tersebut. Anda harus secara eksplisit memanggil fungsi `notifyDataSetChanged()` pada objek *Adapter* agar ia segera kembali mencetak ulang baris-baris tampilan visual yang baru ke layar.

Java.lang.NullPointerException: Attempt to invoke virtual method 'android.view.View android.view....

```
java.lang.NullPointerException: Attempt to invoke virtual
method 'android.view.View android.view.View.findViewById(int)'
on a null object reference...
    at MainActivity.onCreate(MainActivity.java:50)
```

ListView gagal memperbarui tampilan.



ListView gagal memperbarui tampilan. *ArrayList* dimodifikasi langsung tanpa melalui Adapter. Solusi: Panggil `adapter.notifyDataSetChanged()`.

Gemini Notebook

Kesalahan Umum & Solusinya

Gejala	Penyebab Umum	Solusi
App crash saat getView() dipanggil	Salah id pada findViewById vs item_kontak.xml	Periksa ejaan R.id di kode vs XML
Kontak baru tidak muncul di list	Lupa memanggil kAdapter.add(...)	Pastikan simpanKontak() benar-benar dipanggil dari tombol Simpan
Data hilang tiap app ditutup	Hanya menambah ke ArrayList, tidak insert ke DB	Tambahkan dbKu.insert(...) sebelum kAdapter.add(...)
Kontak lama muncul dobel saat dibuka ulang	muatDataDariDatabase() dipanggil lebih dari sekali	Pastikan hanya dipanggil sekali di onCreate()

Tantangan Lanjutan (Opsional)

Aplikasi Buku Kontak yang dibangun kini bisa menambah dan melihat data. Tiga kemampuan lain yang belum dibahas, sengaja disisakan sebagai latihan mandiri:

- Edit: bagaimana membuka dialog yang sudah terisi data lama, lalu meng-UPDATE baris di SQLite yang sesuai?
- Delete: bagaimana menghapus satu kontak dari ListView sekaligus dari tabel kontak (DELETE FROM ...)?
- Cari: bagaimana menyaring ListView agar hanya menampilkan kontak yang cocok dengan kata kunci pencarian?

Petunjuk arah: ketiganya butuh cara mengetahui item mana yang disentuh pengguna di dalam ListView — coba telusuri setOnItemClickListener() dan WHERE _id = ? pada query SQL sebagai titik awal.



Tantangan Mingguan: Skenario Capstone

Aplikasi Buku Kontak Terintegrasi SQLite

- Deskripsi Proyek: Anda ditugaskan membangun aplikasi pencatatan kontak komprehensif yang memadukan UI ListView dinamis (materi Pertemuan 5) dengan penyimpanan data persisten lokal SQLite (materi Pertemuan 4).
- Aturan Desain Utama: Struktur layout XML wajib dirancang dengan rapi. Antarmuka harus benar-benar bebas dari bentrokan atau *overlap* antar widget.
- Penerapan safe zones (*padding* dan *margin*) adalah wajib sehingga teks atau elemen interaktif tidak pernah menabrak bingkai perangkat, terutama pada simulasi layar berukuran kecil.

Gemini Notebook

Kebutuhan Fungsional Mini Project

Kebutuhan UI / Antarmuka

- Menggunakan *Custom ListView* (minimal harus memuat dua baris teks: Nama dan No HP).
- Terdapat komponen tombol tambah (melayang atau statis) yang akan memunculkan jendela *Dialog* kustom saat diklik.
- Menerapkan *Dialog Konfirmasi* ganda sebagai lapisan keamanan sebelum pengguna diizinkan untuk menghapus data kontak (opsional pemicu: lewat aksi klik lama / *LongClick* pada baris list).

Integrasi SQLite / Backend Lokal

- Sistem aplikasi dilarang keras hanya menyimpan data di dalam memori array sementara (RAM).
- Seluruh mutasi data (operasi Tambah, Hapus, Edit) wajib langsung tersinkronisasi dan tertulis ke dalam *database* SQLite.
- Validasi mutlak: Saat aplikasi dimatikan paksa (*Force Close*) dan dibuka kembali oleh pengguna, seluruh data kontak terakhir wajib muncul kembali tanpa ada yang hilang.

Gemini Notebook

Rubrik Penilaian **Capstone Project**

30% | Fungsionalitas Dasar

CRUD SQLite berjalan normal tanpa *crash*, integritas data bertahan setelah *restart* aplikasi.

30% | Kualitas UI & Adapter

Implementasi *Custom ListView*, akurasi *ViewHolder Pattern*, dan desain *Dialog* yang rapi serta bebas *overlap*.

20% | Kemampuan Demonstrasi

Mahasiswa mampu menjelaskan alur kerja aplikasi dan memodifikasi blok kode dengan logis saat sesi *live demo*.

15% | Kualitas Kode

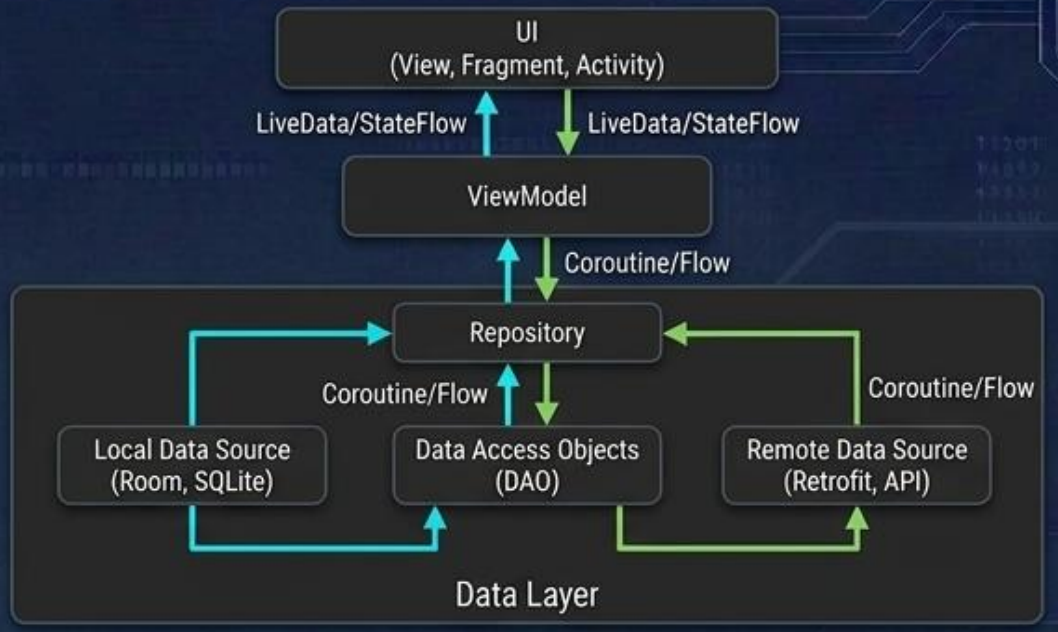
Struktur kode bersih (*clean code*), pemisahan fungsionalitas antar *class* dengan baik (Model, Adapter, Helper).

5% | Ketepatan Waktu

Penyerahan tugas proyek akhir sesuai *deadline* sistem yang telah ditentukan di kelas maya.

Gemini Notebook

Sintesis: Siklus Data End-to-End



Dialog (Input Mentah) ➡ Layer Memori & SQLite (Data Permanen) ➡ Adapter (Deteksi Mutasi) ➡ Render Ulang XML ➡ ListView (Visual Interaktif)

Seluruh arsitektur yang kita bahas beroperasi dalam satu siklus linier dan sinkron. Dialog bertugas menangkap input mentah langsung dari tangan pengguna. Data mentah tersebut diteruskan ke layer memori dan disimpan secara permanen dalam SQLite agar tidak pernah terhapus. Selanjutnya, *Adapter* mendeteksi mutasi data tersebut dan menjembataninya dengan merender ulang baris XML, menampilkannya kembali ke permukaan `ListView` sebagai visual interaktif.

Gemini Notebook

Rangkuman Pertemuan 5

- ListView + ArrayAdapter: pola standar menampilkan data yang banyak & dinamis di Android.
- View Recycling: hanya baris yang terlihat di layar yang dibuat, sisanya didaur ulang — efisien untuk data besar.
- Custom Adapter + ViewHolder Pattern: menampilkan objek kompleks (bukan sekadar teks) dengan performa yang tetap baik.
- Input Dialog (AlertDialog + LayoutInflater): cara standar meminta data baru dari pengguna tanpa berpindah layar.
- Berhasil dibangun Buku Kontak Sederhana — ListView dinamis, dialog input, dan data tersimpan permanen di SQLite.

Rangkuman Fase 1 — Fondasi Android Studio

Lima pertemuan terakhir membentuk satu kesatuan Fase 1: Fondasi Android Studio:

- P1: Instalasi, IDE, dan anatomi Activity pertama.
- P2: Layout XML, widget dasar, dan menangani interaksi pengguna.
- P3: Berpindah antar layar dengan Intent, mengirim & menerima data.
- P4: Menyimpan data dengan SharedPreferences & SQLite.
- P5: Menampilkan data dinamis (ListView), meminta input (Dialog), menyatukan semuanya jadi satu aplikasi nyata.

Kini aplikasi Android sederhana yang lengkap dapat dibangun: multi-layar, merespons interaksi, dan menyimpan data secara permanen — menggunakan pendekatan klasik View/XML berbasis Java.

Menuju Fase 2: Kotlin & Jetpack Compose

Seluruh kode pada pertemuan ini ditulis dengan gaya imperatif: findViewById() dipanggil secara manual, ArrayAdapter diatur, getView() ditulis, dan daur ulang View dikelola lewat ViewHolder.

Mulai pertemuan depan, materi memasuki Fase 2, dengan dua pergeseran besar:

- Kotlin: bahasa yang lebih ringkas dari Java, dengan null safety bawaan (mengurangi crash NullPointerException) — akan terasa familiar karena konsep OOP-nya sama.
- Jetpack Compose: UI deklaratif — cukup dideskripsikan "seperti apa seharusnya tampilan untuk data ini", dan framework yang mengurus pembaruan tampilannya. Tidak ada lagi ArrayAdapter, ViewHolder, atau findViewById() manual.

Ini bukan mata kuliah baru — konsep Android yang telah dikuasai (Activity, Intent, penyimpanan data) tetap berlaku, hanya cara menuliskannya yang berevolusi menjadi lebih modern dan ringkas.

Tabel Perbandingan: Cara Lama vs Cara Baru

Aspek	Fase 1 (Java + XML, P1–P5)	Fase 2 (Kotlin + Compose, mulai P6)
Bahasa	Java	Kotlin
Tampilan	Layout XML terpisah	Fungsi @Composable di kode Kotlin
Ambil elemen UI	findViewById() manual	Tidak perlu — UI "tahu" datanya sendiri
Daftar data	ListView + ArrayAdapter + ViewHolder	LazyColumn — daur ulang ditangani otomatis
Update tampilan	Panggil notifyDataSetChanged()/add()	Ubah state → UI otomatis menyesuaikan
Konsep dasar (Activity, Intent, data)	Berlaku	Tetap berlaku



Pratinjau Pertemuan 6: Transisi ke Kotlin Modern & Fundamental Bahasa

Minggu depan, materi dimulai dari dasar bahasa Kotlin:

- Variabel (val vs var), tipe data, dan null safety
- Fungsi, parameter, dan return value ala Kotlin
- Kontrol alur & struktur data fundamental
- Sesi latihan langsung menulis & menjalankan kode Kotlin pertama

Persiapan: tidak perlu instalasi baru — Android Studio yang sudah terpasang mendukung Kotlin secara langsung.